

Bidirectional Heuristic Search on Voxel Maps

Background from the Literature and a Project Plan

Search Methods in Artificial Intelligence — Ben-Gurion University

Advisor: Lior Siag

July 21, 2026

Abstract

This document distills the six theory papers and one domain paper assigned by Lior Siag into exactly what is needed to start the project: run bidirectional heuristic search algorithms (A^* , MM, BAE*, BiA*, GBFS, plus unidirectional variants) on the Warthog 3D voxel-map benchmarks inside the HOG2 framework, and measure how close each algorithm comes to the theoretical minimum number of node expansions. It is organized so the theory builds up in the order Lior recommended — the “must-expand” foundation first, then the lower-bound machinery, then the concrete algorithms — and closes with the voxel domain details that drive the loader, the experimental methodology, and a recommended two-person workflow.

Contents

1	Project in one page	2
2	Background and notation	2
3	The must-expand theory: which nodes are unavoidable	2
3.1	The must-expand <i>pair</i> (Eckerle et al., 2017)	3
3.2	The must-expand graph and minimum vertex cover (Chen et al., 2017)	3
3.3	Hitting the floor: fractional MM (Shaham et al., 2017)	3
3.4	What consistency buys (Shaham et al., 2018)	4
4	Unifying the lower bounds (Siag and Shperberg, 2025)	4
5	The algorithms we will run	5
5.1	MM — guaranteed to meet in the middle (Holte et al., 2017)	5
5.2	BAE* — bidirectional A^* with error (Alcázar et al., 2020; Siag and Shperberg, 2025)	6
5.3	GBFS — greedy best-first	6
6	The voxel domain (Nobes et al., 2023)	6
7	What to expect on our maps	7
8	Recommended workflow and two-person split	7
8.1	Phased plan	7
8.2	Responsibility split	8
8.3	Ways of working	9
	Reading order	9

1 Project in one page

We are searching for a least-cost path of cost C^* between a **start** and a **goal** state. *Bidirectional heuristic search* (BiHS) runs two searches at once: a **forward** search from **start** and a **backward** search from **goal** (using reversed moves). The promise is that two searches, each travelling “half the distance,” can expand far fewer nodes than a single unidirectional A^* (Holte et al., 2017).

The concrete goal (per Lior’s brief) is to **connect the Warthog voxel-map benchmark to the bidirectional-search code in HOG2**, then run every required algorithm from both directions and analyze the results. HOG2 already contains the algorithms (`MM.h`, `BAE.h`, `TemplateAStar.h`, `NBS.h`, `BidirectionalGreedyBestFirst.h`) and a voxel domain (`Voxels.{h,cpp}`), so the real work is (a) a loader from the Warthog voxel format into HOG2’s domain, and (b) an experiment driver that runs all algorithms both ways and records metrics.

Everything in the assigned papers answers two questions:

1. **Theory:** which nodes must *any* correct algorithm expand? This gives a provable lower bound — the yardstick we compare against (§3–§4).
2. **Algorithms:** how do MM, BAE*, A^* , BiA* and GBFS try to reach that minimum, and when does each win? (§5).

2 Background and notation

Learn this notation first — it appears in every paper.

- $g_F(n)$: cost of the best known path **start**→ n (forward). $g_B(n)$: cost of the best known path n →**goal** (backward).
- $h_F(n)$: heuristic estimate of n →**goal**. $h_B(n)$: estimate of **start**→ n .
- $f_F(n) = g_F(n) + h_F(n)$ and $f_B(n) = g_B(n) + h_B(n)$, exactly as in A^* .
- C^* : optimal solution cost. U : cost of the best (incumbent) solution found so far. ε : the cheapest edge cost in the graph.

Front-to-end vs. front-to-front (know the difference)

All algorithms in this project are **front-to-end**: each side’s heuristic points at the *fixed opposite endpoint* (h_F aims at **goal**, h_B aims at **start**). This contrasts with **front-to-front** heuristics that estimate the distance between two *arbitrary* frontier nodes — more informative but rarely practical (Eckerle et al., 2017). The whole must-expand theory below is stated for the front-to-end setting.

Admissible vs. consistent. A heuristic is *admissible* if it never overestimates ($h_F(n) \leq d(n, \text{goal})$). It is *consistent* if it additionally satisfies the triangle inequality $h_F(n) \leq d(n, n') + h_F(n')$. **Our voxel heuristics (3D-octile / Euclidean distance) are consistent**, not merely admissible — and consistency unlocks tighter lower bounds (§3.4), so it is the regime we actually live in.

3 The must-expand theory: which nodes are unavoidable

This is the intellectual spine, developed across Eckerle et al. (2017), Chen et al. (2017), and the two Shaham papers (Shaham et al., 2017, 2018). Understand it and the rest follows.

3.1 The must-expand *pair* (Eckerle et al., 2017)

In unidirectional A^* the obligation is on *single nodes*: A^* must expand every node with $f < C^*$ (Dechter and Pearl, 1985). Bidirectional search is different — the obligation is on **pairs** (u, v) , where u is a forward-frontier node and v a backward-frontier node.

Define the lower bound on the cost of any path forced through *both* u and v :

$$lb(u, v) = \max\{ f_F(u), f_B(v), g_F(u) + g_B(v) \}. \quad (1)$$

The three terms are three independent reasons such a path must cost at least this much: $f_F(u)$ is the forward estimate of a solution through u ; $f_B(v)$ is the backward estimate through v ; and $g_F(u) + g_B(v)$ is the *meeting term* — the guaranteed cost of reaching u from the start plus reaching the goal from v , *even if the connecting piece were free*. The meeting term is the uniquely bidirectional ingredient.

The rule

(u, v) is a **must-expand pair** iff $lb(u, v) < C^*$. The obligation is discharged by expanding *either* u forward *or* v backward — not necessarily both.

Why (intuition). An expansion-based, black-box algorithm cannot “see” edges it has not generated. If it skips both u and v , an adversary can insert one cheap edge $u \rightarrow v$ that creates a solution cheaper than C^* ; the algorithm never notices and returns a sub-optimal answer. So to be *guaranteed* optimal it must touch at least one side of every must-expand pair. Eckerle et al. (2017) also show this generalizes the classic A^* result (Dechter and Pearl, 1985), which is the special case $h_B = 0$.

3.2 The must-expand graph and minimum vertex cover (Chen et al., 2017)

Turn this into a bipartite graph G_{MX} : forward nodes on the left, backward nodes on the right, an edge between u and v iff (u, v) is a must-expand pair. Because expanding a node “covers” every edge touching it, and every must-expand edge must be covered:

The theoretical floor

The minimum number of expansions any BiHS algorithm can achieve equals the **Minimum Vertex Cover (MVC)** of G_{MX} . Since G_{MX} is bipartite, the MVC is computable in polynomial time (König’s theorem / max-flow), so this floor is *measurable per instance* — it is the denominator we report every algorithm against.

Chen et al. (2017) also introduced **NBS** (Near-optimal Bidirectional Search), which provably expands at most $2 \times \text{MVC}$ and is worst-case optimal — our natural near-optimal baseline.

3.3 Hitting the floor: fractional MM (Shaham et al., 2017)

Shaham et al. (2017) show the MVC need not be built explicitly: cluster nodes by g -value, and the optimal cover is always a clean split — all lowest- g forward nodes up to a cutoff, plus all lowest- g backward nodes up to a dual cutoff — computable in time linear in the number of distinct g -values. They introduce **fMM**(p) (“fractional MM”), which meets at fraction p of the path: $p=1$ is forward A^* , $p=0$ is reverse A^* , $p=\frac{1}{2}$ is MM. There is an optimal p^* for which **fMM**(p^*) expands *exactly* the MVC — but p^* depends on C^* and the instance, so it is only knowable *after* solving. This is the permanent gap between the theoretical optimum and any real algorithm.

Experimental consequence

The best meeting point is domain-dependent and *usually not the middle*. Do not assume MM’s $C^*/2$ split is optimal on voxel maps — measure the distribution of p^* .

3.4 What consistency buys (Shaham et al., 2018)

Because our heuristics are *consistent*, the heuristic difference itself becomes a free front-to-front lower bound on the “gap” $d(u, v)$ between the forward node u and the backward node v . Define

$$h_C(u, v) = \max\{h_F(u) - h_F(v), h_B(v) - h_B(u), \varepsilon\}, \quad (2)$$

and the must-expand condition tightens to $g_F(u) + g_B(v) + h_C(u, v) < C^*$. The two heuristic terms are directional mirror images that each lower-bound the *same* forward gap $d(u, v)$: consistency of h_F gives $h_F(u) - h_F(v) \leq d(u, v)$, and consistency of h_B along the same $u \rightarrow v$ segment gives $h_B(v) - h_B(u) \leq d(u, v)$. (On an *undirected* graph such as a voxel map, where $d(u, v) = d(v, u)$, both signs hold and the terms may be replaced by absolute values $|h_F(u) - h_F(v)|, |h_B(u) - h_B(v)|$ — the “ lb_{CU} ” variant; on directed graphs only the signed form is admissible.) The extra non-negative term makes the inequality fail more often, so *fewer pairs are must-expand* $\Rightarrow$ sparser $G_{MX} \Rightarrow$ smaller floor. In plain terms, consistency lets you *prove* more pairs are safe to skip, which is precisely what consistency-exploiting algorithms (BAE*) capitalize on. Shaham et al. (2018) prove a matching optimal algorithm exists (per heuristic-equivalence class), confirming the lower bound is tight.

4 Unifying the lower bounds (Siag and Shperberg, 2025)

Siag and Shperberg (2025) — Lior’s own capstone paper, and the recommended starting read — proves that all the practical “lower bound” formulas invented over the years are just different subsets of the single $\max\{\cdot\}$ expression in Eq. (1), and that infinitely many valid bounds can be generated from it. Two derived per-node quantities are central:

$$d_D(n) = g_D(n) - h_{\bar{D}}(n) \quad (\text{heuristic error: how much the opposite heuristic underestimated}), \quad (3)$$

$$b_D(n) = f_D(n) + d_D(n) \quad (\text{the } b\text{-value, BAE*’s priority}). \quad (4)$$

where D is a direction and $\bar{D}$ its opposite. From these come the family of global bounds, all valid lower bounds on C^* ; the most useful in practice are:

- the **g-bound** $g\text{Min}_F + g\text{Min}_B + \varepsilon$;
- the **Kaindl–Kainz bounds** (Kaindl and Kainz, 1997) $f\text{Min}_F + d\text{Min}_B$ and $f\text{Min}_B + d\text{Min}_F$;
- the **b-bound** $\frac{1}{2}(b\text{Min}_F + b\text{Min}_B)$, the one BAE* pushes.

No single bound dominates the others, but the empirical headline is decisive for us:

Practical guidance from Siag & Shperberg (2025)

1. **Default to BAE*** under consistent heuristics — best balance of node expansions and run-time.
2. Existing algorithms already come within $\sim 5\text{--}15\%$ of the MVC floor, so there is little room left to reduce expansions by inventing new bounds.
3. **On grid/map-like domains the consistency benefit is small**, and reverse A^* sometimes beats everything because of directional asymmetry. Whether the fancy bidirectional algorithm wins on voxel maps is an *empirical* question — which is exactly what this project answers.

5 The algorithms we will run

Algorithm	Type	Priority / key idea	Optimal?
A^* (+ reverse)	unidirectional	expand min $f = g + h$	yes
MM	bidirectional	expand min $pr = \max(f, 2g)$ (meet in the middle)	yes
BiA*	bidirectional	bidirectional A^* , min f each side; frontiers not capped	yes
BAE*	bidirectional	expand min $b = f + d$, $d = g - h_{\text{opp}}$; exploits heuristic error	yes
GBFS	greedy	expand min h	no
NBS	bidirectional	near-optimal; $\leq 2 \times \text{MVC}$ (baseline)	yes

5.1 MM — guaranteed to meet in the middle (Holte et al., 2017)

The whole trick is the priority function

$$pr_F(n) = \max(f_F(n), 2g_F(n)) \quad (\text{backward symmetric}). \quad (5)$$

A node is expanded only when its priority $\leq C^*$, so the $2g$ term forces $g_F(n) \leq C^*/2$: *neither frontier ever ventures past the halfway distance from its own root*. (This is about how far each search reaches, not physically where they meet.) Implementation essentials:

- two full search states (Open/Closed/ g per direction); backward uses reversed moves;
- each step expands whichever direction owns the global minimum priority $C = \min(pr_{\min_F}, pr_{\min_B})$;
- **solution detection**: when a generated node already sits on the opposite Open list, a full path exists — update $U \leftarrow \min(U, g_F(n) + g_B(n))$;
- **termination**: stop when $U \leq \max(C, f_{\min_F}, f_{\min_B}, g_{\min_F} + g_{\min_B} + \varepsilon)$; *not* at first frontier contact (that yields sub-optimal answers);

- **MM0** ($h=0$, expand min $2g$) is the bidirectional brute-force baseline, analogous to Dijkstra.

MM wins with moderate-strength heuristics; a very strong heuristic favors A^* , a very weak one favors MM0.

5.2 BAE* — bidirectional A^* with error (Alcázar et al., 2020; Siag and Shperberg, 2025)

BAE* is a bidirectional best-first search whose priority is the b -value $b_D(n) = f_D(n) + d_D(n) = f_D(n) + (g_D(n) - h_{\bar{D}}(n))$, terminating on the b -bound. Intuitively it “charges the heuristic’s looseness back into” a node’s priority, delaying loose nodes. Under consistency b stays monotone, so BAE* is optimal. The implementation delta from BiA* is small (swap the priority, use the b -bound to stop), and it is Lior’s recommended default. Alcázar et al. (2020) showed it can expand up to an order of magnitude fewer “necessarily expanded” nodes than near-optimal algorithms such as NBS on some domains, because exploiting heuristic inaccuracy turns many would-be must-expand nodes into delayable ones.

5.3 GBFS — greedy best-first

GBFS expands by lowest h only and is **not cost-optimal**; none of the must-expand/MVC theory applies to it. Report it on a separate axis (fast, but its path cost and expansions are not comparable against the MVC floor).

6 The voxel domain (Nobes et al., 2023)

This is the practical layer the loader must match.

What a voxel map is. A 3D integer lattice width $\times$ height $\times$ depth; each voxel is filled (obstacle) or free, with up to 26 neighbours (the 3D analogue of the 8-neighbour 2D grid). The benchmark ships three families: **Sandstone** (dense porous rock, all 400^3 , $\sim 80\%$ filled), **Descent** (1995 game levels, rooms + corridors, up to ~ 1.3 billion voxels), and **Industrial Plants** (pipe-routing layouts, sparse). In total: 46 maps, 92,000 instances.

File format — the single most error-prone step

```
voxel          <- OR "rev_voxel"
<width> <height> <depth>
x0 y0 z0
x1 y1 z1
...
```

voxel $\Rightarrow$ listed coordinates are the *filled/obstacle* voxels (everything else free). **rev_voxel** $\Rightarrow$ listed coordinates are the *free* voxels (everything else blocked) — used for dense maps like Sandstone. *Branch on the first token, or dense maps come out completely inverted.* The domain paper names the fields but gives no byte-exact spec: verify indexing order, 0- vs 1-based, whitespace, and scenario columns against a real downloaded file and the Warthog loader source before trusting a parser.

Movement model (match exactly for comparable costs).

- **26-connected** moves with 3D-octile costs: straight (face) = 1, edge-diagonal = $\sqrt{2}$, corner-diagonal = $\sqrt{3}$. Hence $\epsilon = 1$.
- **No corner-cutting:** a diagonal move is legal only if the voxels it clips are all free.
- Region *reachability* is computed with 6-connectivity (flood fill); keep that distinct from the 26-connected *move* model.
- “Without diagonals” (per the brief) = the 6 face moves at unit cost; “with diagonals” = full 26-connected. Since the domain supports diagonals, we run both.

Scenario files and metrics. Each map ships $\sim 2,000$ **start/goal** voxel triples, stored in the minimum-cost direction (which controls directional bias — directly relevant when validating bidirectional symmetry), likely carrying an optimal cost. The currency of comparison is **node expansions**. Report two flavors used by the papers: expansions to *prove* optimality (bound $< C^*$) versus total ($\leq C^*$), each as a ratio to the per-instance MVC floor. Also log runtime and solution cost — all optimal algorithms must return the same cost as A^* , which is our correctness check.

7 What to expect on our maps

Synthesizing the empirical findings across the papers, on grid/voxel-like domains:

- A^* (and reverse A^*) are surprisingly competitive; strong Euclidean/octile heuristics narrow the bidirectional advantage (Holte et al., 2017; Siag and Shperberg, 2025).
- MM/BAE* pay off most with moderate heuristics; the consistency benefit is modest on maps (Shaham et al., 2018; Siag and Shperberg, 2025).
- The best meeting point is rarely the exact middle; directional asymmetry matters, so always run *both* directions (Shaham et al., 2017; Nobes et al., 2023).
- GBFS is fast but sub-optimal — separate axis.
- Report every optimal algorithm as a ratio to the MVC floor (or to NBS’s $\leq 2 \times \text{MVC}$ line) rather than as raw counts.

8 Recommended workflow and two-person split

The work factors cleanly into an **infrastructure track** (get maps into HOG2 and run things) and an **algorithms/analysis track** (make all algorithms run both ways and turn numbers into findings). We propose **Person A = Infrastructure** and **Person B = Algorithms & Analysis**, with shared milestones and paired reviews at each phase boundary.

8.1 Phased plan

Ph	Milestone	Deliverable / exit criterion	Owner
----	-----------	------------------------------	-------

0	Setup & read	HOG2 built; benchmarks fetched; both read papers 2, 3, 1, 6	Both
1	Format spike	One real Sandstone (<code>rev_voxel</code>) + one Descent (<code>voxel</code>) map parsed and visualized; format documented	A (lead), B (review)
2	Loader	Warthog $\rightarrow$ HOG2 Voxels loader; 26-conn + no-corner-cutting + octile costs; unit tests	A
3	Baseline run	A^* and reverse A^* produce optimal costs matching benchmark; metrics logged (expansions, runtime, cost)	B (lead), A (loader support)
4	Algorithm sweep	MM, MM0, BiA*, BAE*, GBFS, NBS wired into the driver; run both directions, with and without diagonals	B
5	MVC yardstick	Per-instance MVC of G_{MX} computed; expansion ratios produced	B (lead), A (perf/IO)
6	Scale & robustness	Large Descent/Plant maps stream without OOM; timeouts/memory caps handled	A
7	Analysis & report	Tables + plots per family; crossover discussion; final write-up	Both

8.2 Responsibility split

Person A — Infrastructure			Person B — Algorithms & Analysis
Warthog	voxel loader	scenario	Experiment driver looping all algorithms $\times$ both directions $\times$ diagonal on/off
HOG2 Voxels domain wiring: 26-connectivity, no-corner-cutting, octile costs			Configure/verify MM, MM0, BiA*, BAE*, GBFS, NBS in HOG2
Bit-packed storage & streaming for billion-voxel maps; build system			Metrics: expansions ($< C^*$ and $\leq C^*$), runtime, cost, meeting point

Correctness & perf harness; time-outs, memory caps, reproducible runs	MVC-of- G_{MX} computation as the theoretical floor; ratio reporting
CI-style regression on a small map subset	Statistical analysis, plots, and the results tables

8.3 Ways of working

- **Contract early.** Agree the loader’s output data structure (the occupancy grid + domain interface) in Phase 1 so A and B can work in parallel behind it. B can start on a tiny hand-made map before the real loader lands.
- **Correctness gate.** No algorithm result is trusted until its returned cost equals A^* ’s on the same instance (Phase 3 establishes the oracle).
- **Reference implementation.** Mirror the SPL-BGU BiHS-Direction-Choosing usage example and the BiHS-Consistent-F2E repo for how HOG2’s BiHS code is driven.
- **Cross-review at phase boundaries.** Each phase’s exit criterion is reviewed by the other person; this catches the two classic bugs early — inverted `rev_voxel` polarity, and corner-cutting sneaking in cheaper-than-optimal paths.
- **Scale last.** Develop on Sandstone (400^3) and small Descent maps; only move to billion-voxel maps once the pipeline is correct and instrumented.

Reading order (Lior’s recommendation)

Start with Siag and Shperberg (2025) (the unifying big picture); if clear, continue to Eckerle et al. (2017) (the must-expand foundation). Then Holte et al. (2017) (MM — the most concrete algorithm), Alcázar et al. (2020) (BAE*, the likely default), and Shaham et al. (2017, 2018) for the minimal-set and consistency depth. Read Nobes et al. (2023) alongside the loader work.

References

- Alcázar, V., Riddle, P., and Barley, M. (2020). A Unifying View on Individual Bounds and Heuristic Inaccuracies in Bidirectional Search. In *Proc. AAAI Conference on Artificial Intelligence*, pp. 2327–2334.
- Chen, J., Holte, R. C., Zilles, S., and Sturtevant, N. R. (2017). Front-to-End Bidirectional Heuristic Search with Near-Optimal Node Expansions. In *Proc. IJCAI*, pp. 489–495.
- Dechter, R. and Pearl, J. (1985). Generalized best-first search strategies and the optimality of A^* . *Journal of the ACM*, 32(3):505–536.
- Eckerle, J., Chen, J., Sturtevant, N. R., Zilles, S., and Holte, R. C. (2017). Sufficient Conditions for Node Expansion in Bidirectional Heuristic Search. In *Proc. ICAPS*.
- Holte, R. C., Felner, A., Sharon, G., and Sturtevant, N. R. (2017). MM: A bidirectional search algorithm that is guaranteed to meet in the middle. *Artificial Intelligence*, 252:232–266.

- Kaindl, H. and Kainz, G. (1997). Bidirectional Heuristic Search Reconsidered. *Journal of Artificial Intelligence Research*, 7:283–317.
- Nobes, R., Harabor, D., Wybrow, M., and Walsh, S. D. C. (2023). Voxel Benchmarks for 3D Pathfinding: Sandstone, Descent, and Industrial Plants. In *Proc. Symposium on Combinatorial Search (SoCS)*.
- Shaham, E., Felner, A., Chen, J., and Sturtevant, N. R. (2017). The Minimal Set of States that Must Be Expanded in a Front-to-End Bidirectional Search. In *Proc. Symposium on Combinatorial Search (SoCS)*.
- Shaham, E., Felner, A., Sturtevant, N. R., and Rosenschein, J. S. (2018). Minimizing Node Expansions in Bidirectional Search with Consistent Heuristics. In *Proc. Symposium on Combinatorial Search (SoCS)*.
- Siag, L. and Shperberg, S. S. (2025). Bridging theory and practice in bidirectional heuristic search with front-to-end consistent heuristics. *Artificial Intelligence*, 348:104420.